Lefèvre's lower bound, and its proof in Rocq

A guide to `AlgFGG.v`, `AlgLefevre.v` and `Config.v` for readers who do not read Rocq

1 What is being computed

The hard-to-round search needs, for a line $y = ax - b$ over an interval of N points, a **lower bound** on

$$\inf\{b - ax \bmod 1 : x < N\}.$$

If that bound exceeds a threshold, no point of the interval can be a hard-to-round case and the interval is discarded without further work. Lefèvre's algorithm computes such a bound in $O(\log)$ steps rather than $O(N)$.

Everything is scaled by a modulus M so that the development is over natural numbers: $a = A/M$, $b = B/M$. Two functions carry the geometry:

- `pt x` $= (Ax) \bmod M$, the x-th point of the orbit;
- `dst x` $= (B + M - $ `pt x`$) \bmod M$, its distance **down** to b;

and `inf_dst M A B n`, written $\mathrm{Inf}(n)$ below, is the minimum of `dst x` over $x < n$. The theorem to prove is that the algorithm's result is at most $\mathrm{Inf}(N)$.

There are two algorithms, and **they are presented here in the reverse of their numbering**. Algorithm 1 is Lefèvre's original; Algorithm 2 is the later, batched variant designed to replace it. But Algorithm 2′s development came first and is what Algorithm 1′s proof reuses throughout, so it is described first below. Where the numbering could mislead, the sections are named for what the loop does rather than for its number.

2 The structure the algorithm walks

2.1 Two-length configurations

Take the first n points of the orbit and cut the circle at them. The **three-distance theorem** says at most three gap lengths occur, and for the values of n the algorithm visits, exactly two: p and q. The state carries those two lengths together with how many gaps of each there are, u and v. This is the record `inv`, whose essential clauses are

```
u * p + v * q = M     p = pt v     q = M - pt u
```

The first says the gaps tile the circle; the other two say the two lengths are themselves points of the orbit. A second record, `invx`, fixes the **index** structure — which point succeeds which:

```
z < u ⟹ pt (z + v) = pt z + p     u <= z < u + v ⟹ pt (z - u) = (pt z + q) mod M
```

So an index below u heads a gap of length p, and an index at least u heads one of length q. That single fact is what lets the proofs say "b is in a p-gap" without any geometry: it means the index of the point just below b is smaller than u.

2.2 One step

Passing from n points to more points is a Euclidean reduction on the pair (p, q). Reducing the larger gap q by k copies of p splits each q-gap and updates the counts:

```
q -= k*p, u += k*v        or        p -= k*q, v += k*u
```

The paper's Property 3 says these splits are **directional**: a q-gap becomes k gaps of length p followed by the residual, left to right, whereas a p-gap becomes the residual followed by k gaps of length q — points enter from the right. That asymmetry is visible throughout the proofs.

`Config.v` proves what one reduction does to a configuration, **for any k up to the quotient**: it preserves `inv` (`inv_red_lt`, `inv_red_ge`) and `invx` (the `invx_red_*` families), and it moves the infimum by a known amount. Algorithm 2 always takes k maximal; Algorithm 1 takes it maximal on one side of a turn and $k = 1$ on the other, so both are instances of the same lemmas. The maximal case is named `step` there, and its lemmas — `step_bez`, `step_p_gt0`, `inv_step_pos`, `inf_new_eq_lt` and `invx_step` — are the general ones with k taken to be the quotient. `AlgFGG.v`'s loop is `step` itself; `AlgLefevre.v` reduces the first half of its turn to it.

3 The two listings

Both are transcribed from §4.1. p and q are the two gap lengths, u and v their counts, d the recorded distance, and ε'' the threshold.

Algorithm 1 (Lefèvre). Two reductions per turn, and the exit test between them.

```
 1  p <- {a}; q <- 1 - {a}; d <- {b}; u <- 1; v <- 1
 2  if d < eps then return Failure
 3  while True do
 4    if d < p then
 5      k <- q / p
 6      q <- q - k*p;  u <- u + k*v
 7      if u + v >= N then return Success
 8      p <- p - q;    v <- v + u
 9    else
10      d <- d - p
11      if d < eps then return Failure
12      k <- p / q
13      p <- p - k*q;  v <- v + k*u
14      if u + v >= N then return Success
15      q <- q - p;    u <- u + v
```

Line 13 prints in the paper as `q <- p - k*q`; that has to be a typo, since it would leave p at its old value and send line 15 negative.

Algorithm 2 (Fortin–Gouicem–Graillat). One reduction per turn, the branch on the two lengths, and a single exit test.

```
 1  p <- {a}; q <- 1; d <- {b}; u <- 1; v <- 1
 2  if d < eps then return Failure
 3  while True do
 4    if p < q then
 5      k <- q / p
 6      q <- q - k*p
 7      u <- u + k*v
 8      d <- d mod p
 9    else
10      k <- p / q
11      p <- p - k*q
12      v <- v + k*u
13      if p <= d then d <- (d - p) mod q
14    if u + v >= N then return Success
```

In the development the loops are `run` and `run1`, driven by structural fuel rather than `while True`, and the early `Failure` exits are dropped: d never increases, so testing the returned value against ε'' is equivalent.

4 Algorithm 2: one reduction per turn

4.1 The loop

One turn is one reduction, chosen by comparing the two lengths, together with an update of a recorded distance d:

$$\texttt{step p q d u v} = \text{if } p < q \text{ then reduce } q \text{ else reduce } p$$

and `run` iterates it until $u + v$ reaches N, returning d.

4.2 Why the result is a lower bound

The interesting variable is d. It is **not** the infimum: batching the reduction makes the loop overshoot, so d can already refer to a point beyond the current count. What is maintained instead is the record `invd`, three clauses:

$$\texttt{d < maxn p q} \qquad \texttt{d <= Inf (u + v)} \qquad d \equiv \mathrm{Inf}(u+v) \pmod{p}$$

— d is below the infimum, and congruent to it modulo the smaller gap. The second clause is what makes the returned value a lower bound; the third is what lets it survive a reduction, because a reduction changes the infimum by a multiple of p.

The main work is showing the three records survive a step. For `inv` and `invx` this is the tiling bookkeeping. For `invd` it is a statement about where the new points land relative to b, and it splits by which gap is being reduced:

- reducing q (`inf_new_lt`): every point walks down by steps of p, so the new infimum is exactly $\mathrm{Inf} \bmod p$ — clean, because the walk can be iterated until it can go no further;
- reducing p (`ge_inf_alt`): points enter p-gaps from the right, so whether the infimum moves depends on which gap holds b. The result is a **disjunction**: either the new infimum is the new d, or it is unchanged and was already below q.

That asymmetry is not an artefact of the formalisation; it is Property 3.

Two lemmas of `Config.v` do the geometric work and are reused constantly by both developments: `gap_p_empty` and `gap_q_empty` say that a point with b inside its own gap is the nearest point below b — nothing else in range can be closer. A third, `gap_walk`, is the tiling in index form: for any two indices in range whose distances differ by at most a given bound,

$$\texttt{dst y - dst z = a*p + b*q} \quad \text{with} \quad \texttt{a*v + y = b*u + z},$$

and a companion, `gap_bounds`, turns that into $a \le u$ and $b \le v$. Most "no point can be there" arguments are one application of the two.

The soundness theorem is `lefevre_sound`, and the form the search uses is `lefevre_test`: if the returned bound clears ε, then every $x < N$ has `dst x` above ε.

5 Algorithm 1: two reductions per turn

5.1 Why it is not Algorithm 2 with different constants

Algorithm 1 is the original, and it differs from the algorithm just described in two ways that matter to the proof.

First, **one turn is two reductions**: a division step and then a single subtraction. Second — and this is the awkward part — **the exit test sits between them**. The loop tests the count after the division half, and if it does not exit, performs the subtraction half, which adds more points without testing again. Lefèvre says so explicitly: the algorithm stops not at N but at the first configuration size at least N.

So the file has `half1` and `half2`. On the configuration `half1` does what `Config.step` does whenever the branch test $d < p$ agrees with $p < q$ — same quotient, same counts — and leaves it alone otherwise, the quotient then being 0; the two differ in how they update d. `half2` is the same reduction at $k = 1$.

5.2 What d is, and where

Both the paper and the thesis say d is the distance from b to the nearest point on its left. That is true — **between the halves**, which is where the exit test reads d and where the loop returns it. It is not true at the top of a turn, because `half2` has just added points and left d untouched. Algorithm 2′s `invd` does not hold here at all; it already fails at the initial state.

The invariant that does hold at a turn start is the record `invw`:

$$\texttt{d < p + q} \qquad \mathrm{Inf}(u+v) = \texttt{if d < p then d else d - p}$$

In words: d is the infimum plus the one p-step that `half1` has not yet taken — and which of the two cases holds is exactly what the branch test decides. Consequently `half1_exact` gives $d = \mathrm{Inf}$ after the division half, and the returned value is an infimum over a range at least as large as N, which is the bound.

5.3 The third clause

`invw` has a third field, and it is the one place where the formalisation had to add something the sources only assert. §4.1 remarks that

"the condition at line 4 is true if b is in an interval of length p"

which is what makes the six-case analysis work. One direction is immediate: if b is in a p-gap then $d < p$, since d is a distance inside that gap. The converse is **not** a property of the state — when the infimum is below both gap lengths, the configuration alone does not say which gap holds b (`inf_red_ge_le` gives only the two one-way implications). It is a property of the states the loop reaches, kept true by Property 3′s directionality. So it is carried as an invariant clause,

$$\texttt{(y < u) = (d < p)} \quad \text{for the index } y \text{ attaining the infimum,}$$

using `invx_p1`/`invx_p2` to read "index below u" as "p-gap".

5.4 The turn, case by case

With that in place the two halves are:

- **`half1`** does not move the infimum, in either branch. If $d < p$ then d is already the infimum, so the reduction of q cannot lower it (`half1_inf_lt`). If $p \le d$ then b sits in a q-gap, and reducing p splits p-gaps only, so no point enters below b (`half1_ge_nodrop`). What it does is subtract the pending p from d.
- **`half2`** adds points and leaves d alone, so it is where the infimum drops — by nothing, or by the new p — and where the invariant is restored (`invw_sub_p`, `invw_sub_q`). The two sides are not symmetric, again by Property 3: on the q side the two cases agree with the test on their own, while on the p side the lemma must be told that b is in a p-gap.

5.5 Soundness

`run1_sound` is an induction on the fuel. Each turn: `half1_exact` makes d the infimum, the exit case is `half1_leq_inf`, and the recursive case rebuilds the three records. The overshoot — a turn beginning with the count already past N — is closed by `run1_past`, which needs only `invw`: the loop returns at most the d that `half1` leaves, and that is an infimum over a range at least as large as N.

The results are

```
lefevre1_sound : 2 < N -> lefevre1 M A B N <= Inf N
```

and the corollary the search uses, `lefevre1_test`.

6 State of the development

File	Statements	Contents
`Dist.v`	50	`pt`, `dst`, `Inf` and their arithmetic
`Config.v`	60	the configuration `inv`/`invd`/`invx`, the gap and walk lemmas, one reduction at an arbitrary k, and `step` at the quotient
`AlgFGG.v`	46	Algorithm 2: `run`, the `invd` step lemmas, `lefevre_sound`
`AlgLefevre.v`	50	Algorithm 1: `half1`/`half2`, `run1`, `invw`, `lefevre1_sound`

The dependencies are `Dist` → `Config` → {`AlgFGG`, `AlgLefevre`}; the two algorithm files are independent of each other.

Both soundness theorems, and both `_test` corollaries, are proved without axioms: `Print Assumptions` reports **closed under the global context** for each.

Neither algorithm is exact — both can return a bound strictly below the true infimum, and each file records a witness as a computed `Example`. Algorithm 1 is the sharper of the two; that comparison is measured, not proved.

One thing remains open in the organisation rather than the mathematics: several range hypotheses in `AlgFGG.v` are of the form "the count is below N" where "below the orbit length $M/\gcd(A, M)$" would do, which is what `Config.v` uses.

7 Sources

The algorithm is Lefèvre's, and the analysis the proofs follow is Fortin, Gouicem and Graillat's.

[1] P. Fortin, M. Gouicem, S. Graillat, *Correctly rounding elementary functions on GPU*, hal-00751446 (`doc/mourad.pdf`). §3.2 states and proves Properties 1–3 on two-length configurations; §4.1 gives Algorithm 1 with its six cases for d, and Algorithm 2. This is the analysis both developments follow, and the source of the listing transcribed in `AlgLefevre.v`.

[2] V. Lefèvre, *Moyens arithmétiques pour un calcul fiable*, thèse, ENS Lyon, 2000, ch. 2. Says what the variables mean: u and v count the intervals of each length, and d is the distance from the point considered to the lower endpoint of its interval. Also states the overshoot outright — the loop stops not at N but at the first configuration size at least N.

[3] V. Lefèvre, *An Algorithm That Computes a Lower Bound on the Distance Between a Segment and* $\mathbb{Z}^2$, Developments in Reliable Computing, 203–212. The published form of the same algorithm.

[4] V. Lefèvre, J.-M. Muller, *Worst Cases for Correct Rounding of the Elementary Functions in Double Precision*, INRIA RR-4044 (`doc/RR-4044.pdf`). The search this bound serves.

[5] N. B. Slater, *Gaps and steps for the sequence* $n\theta \bmod 1$, Proc. Camb. Phil. Soc. 63 (1967) 1115–1123 (`doc/slater.pdf`). The three-distance theorem; §2 is the form `invx` uses, giving the successor of a point by index rather than by position.

The repository carries reading notes alongside these: `doc/mourad-notes.md` (the six cases and Property 3), `doc/lefevre-these-notes.md` (the variables), `doc/slater-notes.md` (the dictionary to Slater), `doc/alg2-notes.md`.

Two points where the sources stop short of what a proof needs are marked in the text: §4.1′s line-4 remark, which is asserted rather than proved and becomes an invariant clause here (§4.3); and the fact that neither source states a loop invariant, so `invd` and `invw` are of the developments' own making.